

SECTION 2

DATA ORGANIZATION AND ADDRESSING CAPABILITIES

Most external references to memory by a microprocessor are either program references or data references; they either access instruction words or operands (data items) for an instruction. Program references are references to the program space, the section of memory that contains the program instructions and any immediate data operands that reside in the instruction stream. Refer to M68000PM/AD, *M68000 Programmer's Reference Manual*, for descriptions of the instructions in the program space. Data references refer to the data space, the section of memory that contains the program data. Data items in the instruction stream can be accessed with the program counter relative addressing modes, and these accesses are classified as program references. A third type of external reference used for coprocessor communications, interrupt acknowledge cycles, and breakpoint acknowledge cycles is classified as a CPU space reference. The MC68030 automatically sets the function codes to access the program space, the data space, or the CPU space for special functions as required. The function codes can be used by the memory management unit to organize separate program (read only) and data (read-write) memory areas.

This section describes the data organization and addressing capabilities of the MC68030. It lists the types of operands used by instructions and describes the registers and their use as operands. Next, the section describes the organization of data in memory and the addressing modes available to access data in memory. Last, the section describes the system stack and user program stacks and queues.

2.1 INSTRUCTION OPERANDS

The MC68030 supports a general-purpose set of operands to serve the requirements of a large range of applications. Operands of MC68030 instructions may reside in registers, in memory, or within the instructions themselves. An instruction operand might also reside in a coprocessor. An operand may be a single bit, a bit field of from 1 to 32 bits in length, a byte (8 bits), a word (16 bits), a long word (32 bits), or a quad word (64 bits). The operand size for each instruction is either explicitly encoded in the instruction or implicitly

defined by the instruction operation. Coprocessors are designed to support special computation models that require very specific but widely varying data operand types and sizes. Hence, coprocessor instructions can specify operands of any size.

2.2 ORGANIZATION OF DATA IN REGISTERS

The eight data registers can store data operands of 1, 8, 16, 32, and 64 bits, addresses of 16 or 32 bits, or bit fields of 1 to 32 bits. The seven address registers and the three stack pointers are used for address operands of 16 or 32 bits. The control registers (SR, VBR, SFC, DFC, CACR, CAAR, CRP, SRP, TC, TT0, TT1, and MMUSR) vary in size according to function. Coprocessors may define unique operand sizes and support them with on-chip registers accordingly.

2.2.1 Data Registers

Each data register is 32 bits wide. Byte operands occupy the low-order 8 bits, word operands the low-order 16 bits, and long-word operands the entire 32 bits. When a data register is used as either a source or destination operand, only the appropriate low-order byte or word (in byte or word operations, respectively) is used or changed; the remaining high-order portion is neither used nor changed. The least significant bit of a long-word integer is addressed as bit zero, and the most significant bit is addressed as bit 31. For bit fields, the most significant bit is addressed as bit zero, and the least significant bit is addressed as the width of the field minus one. If the width of the field plus the offset is greater than 32, the bit field wraps around within the register. The following illustration shows the organization of various types of data in the data registers.

Quad-word data consists of two long words: for example, the product of 32-bit multiply or the quotient of 32-bit divide operations (signed and unsigned). Quad words may be organized in any two data registers without restrictions on order or pairing. There are no explicit instructions for the management of this data type, although the MOVEM instruction can be used to move a quad word into or out of the registers.

Binary-coded decimal (BCD) data represents decimal numbers in binary form. Although many BCD codes have been devised, the BCD instructions of the M68000 Family support formats in which the four least significant bits consist of a binary number having the numeric value of the corresponding decimal number. Two BCD formats are used. In the unpacked BCD format, a byte

[illegible]

31	24	23	16	15	8	7	0
High-Order Byte		Middle-High Byte		Middle-Low Byte		Low-Order Byte	

31	16	15	0
High-Order Word		Low-Order Word	

31	0
Long Word	

63	62	32
MSB	Any Dx	

31	1	0
Any Dy		LSB

31	Width			0
Offset		MSB	...	LSB

Unpacked BCD (a = MSB)

31	8	7	6	5	4	3	2	1	0
		x	x	x	x	a	b	c	d

31	8	7	6	5	4	3	2	1	0
	a	b	c	d	e	f	g	h	

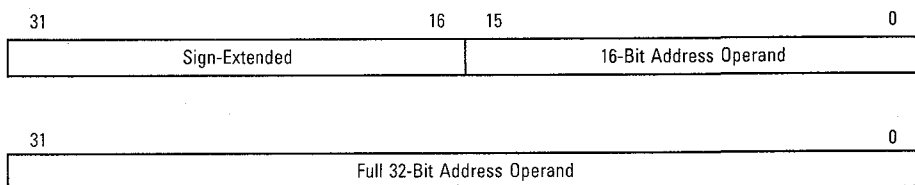
2-3

contains one digit; the four least significant bits contain the binary value and the four most significant bits are undefined. Each byte of the packed BCD format contains two digits; the least significant four bits contain the least significant digit.

2

2.2.2 Address Registers

Each address register and stack pointer is 32 bits wide and holds a 32-bit address. Address registers cannot be used for byte-sized operands. Therefore, when an address register is used as a source operand, either the low-order word or the entire long-word operand is used, depending upon the operation size. When an address register is used as the destination operand, the entire register is affected, regardless of the operation size. If the source operand is a word size, it is first sign-extended to 32 bits and then used in the operation to an address register destination. Address registers are used primarily for addresses and to support address computation. The instruction set includes instructions that add to, subtract from, compare, and move the contents of address registers. The following example shows the organization of addresses in address registers.



Address Organization in Address Registers

2.2.3 Control Registers

The control registers described in this section contain control information for supervisor functions and vary in size. With the exception of the user portion of the status register (CCR), they are accessed only by instructions at the supervisor privilege level.

The status register (SR), shown in Figure 1-4, is 16 bits wide. Only 12 bits of the status register are defined; all undefined values are reserved by Motorola for future definition. The undefined bits are read as zeros and should be written as zeros for future compatibility. The lower byte of the status register is the CCR. Operations to the CCR can be performed at the supervisor or user

privilege level. All operations to the status register and CCR are word-sized operations, but for all CCR operations, the upper byte is read as all zeros and is ignored when written, regardless of privilege level.

The supervisor programming model (see Figure 1-3) shows the control registers. The cache control register (CACR) provides control and status information for the on-chip instruction and data caches. The cache address register (CAAR) contains the address for cache control functions. The vector base register (VBR) provides the base address of the exception vector table. All operations involving the CACR, CAAR, and VBR are long-word operations, whether these registers are used as the source or the destination operand.

The alternate function code registers (SFC and DFC) are 32-bit registers with only bits 2:0 implemented that contain the address space values (FC0–FC2) for the read or write operands of MOVES, PLOAD, PFLUSH, and PTEST instructions. The MOVEC instruction is used to transfer values to and from the alternate function code registers. These are long-word transfers; the upper 29 bits are read as zeros and are ignored when written.

The remaining control registers in the supervisor programming model are used by the memory management unit (MMU). The CPU root pointer (CRP) and supervisor root pointer (SRP) contain pointers to the user and supervisor address translation trees. Transfers of data to and from these 64-bit registers are quad-word transfers. The translation control register (TC) contains control information for the MMU. The MC68030 always uses long-word transfers to access this 32-bit register. The transparent translation registers (TT0 and TT1) also contain 32 bits each; they identify memory areas for direct addressing without address translation. Data transfers to and from these registers are long-word transfers. The MMU status register (MMUSR) stores the status of the MMU after execution of a PTEST instruction. It is a 16-bit register, and transfers to and from the MMUSR are word transfers. Refer to **SECTION 9 MEMORY MANAGEMENT UNIT** for more detail.

2.3 ORGANIZATION OF DATA IN MEMORY

Memory is organized on a byte-addressable basis where lower addresses correspond to higher order bytes. The address, N , of a long-word data item corresponds to the address of the most significant byte of the highest order word. The lower order word is located at address $N + 2$, leaving the least significant byte at address $N + 3$ (refer to Figure 2-1). Notice that the MC68030

does not require data to be aligned on word boundaries (refer to Figure 2-2), but the most efficient data transfers occur when data is aligned on the same byte boundary as its operand size. However, instruction words must be aligned on word boundaries.

The data types supported in memory by the MC68030 are bit and bit field data; integer data of 8, 16, or 32 bits; 32-bit addresses; and BCD data (packed and unpacked). These data types are organized in memory as shown in Figure 2-2. Note that all of these data types can be accessed at any byte address.

Coprocessors can implement any data types and lengths up to 255 bytes. For example, the MC68881/MC68882 floating-point coprocessors support memory accesses for quad-word-sized items (double-precision floating-point values).

A bit operand is specified by a base address that selects one byte in memory (the base byte) and a bit number that selects the one bit in this byte. The most significant bit of the byte is bit 7.

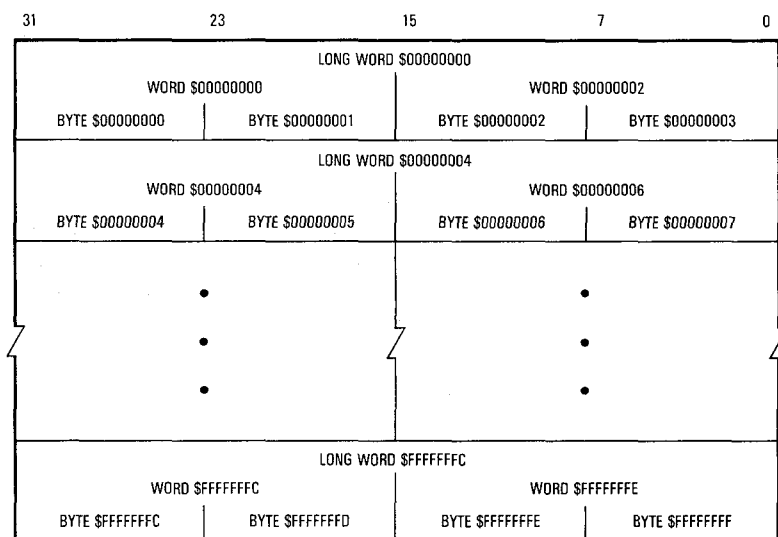


Figure 2-1. Memory Operand Address

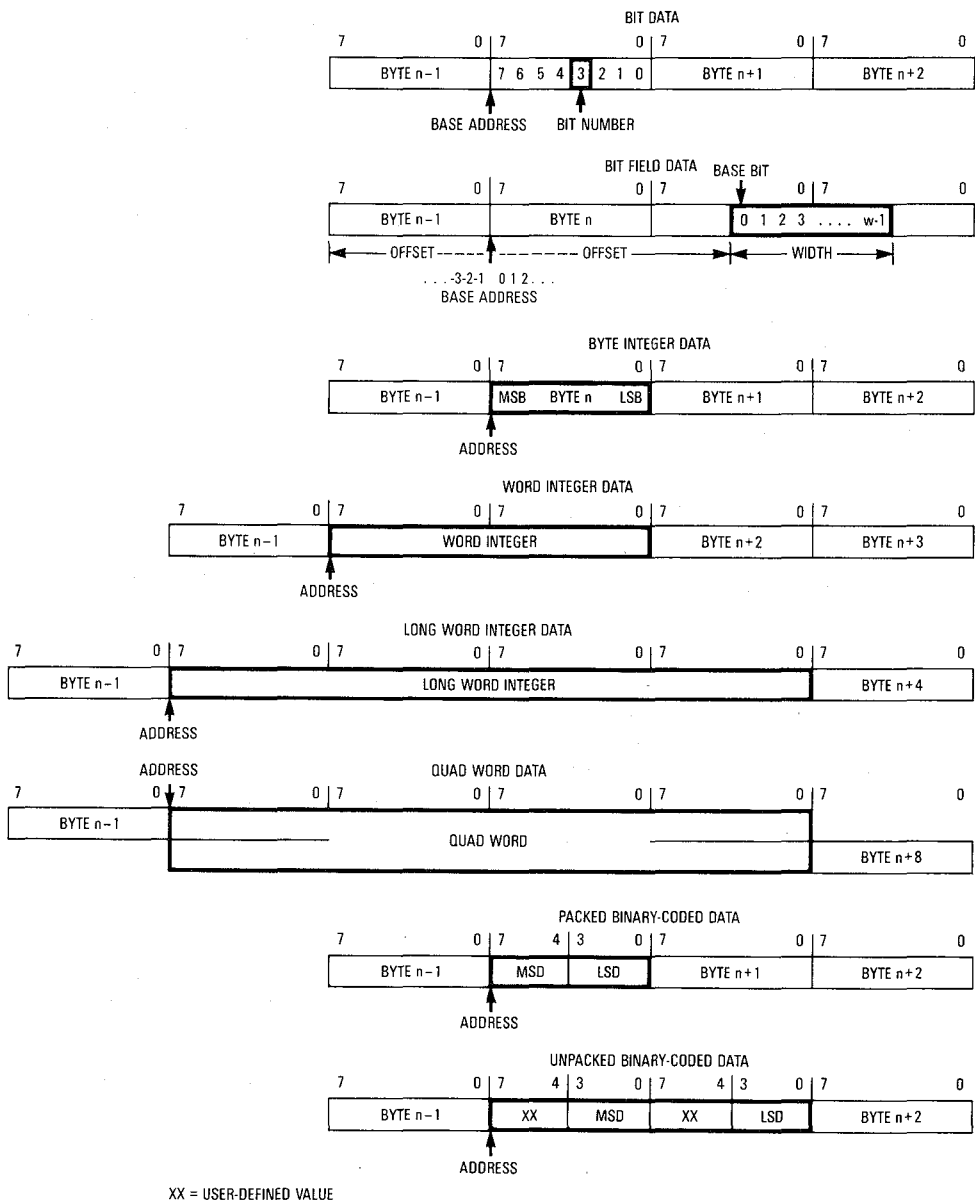


Figure 2-2. Memory Data Organization

A bit field operand is specified by:

1. A base address that selects one byte in memory,
2. A bit field offset that indicates the leftmost (base) bit of the bit field in relation to the most significant bit of the base byte, and
3. A bit field width that determines how many bits to the right of the base bit are in the bit field.

The most significant bit of the base byte is bit field offset 0, the least significant bit of the base byte is bit field offset 7, and the least significant bit of the previous byte in memory is bit offset -1 . Bit field offsets may have values in the range of -2^{31} to $2^{31} - 1$, and bit field widths may range between 1 and 32 bits.

2.4 ADDRESSING MODES

The addressing mode of an instruction can specify the value of an operand (with an immediate operand), a register that contains the operand (with the register direct addressing mode), or how the effective address of an operand in memory is derived. An assembler syntax has been defined for each addressing mode.

Figure 2-3 shows the general format of the single effective address instruction operation word. The effective address field specifies the addressing mode for an operand that can use one of the numerous defined modes. The $\langle ea \rangle$ designation is composed of two 3-bit fields: the mode field and the register field. The value in the mode field selects one or a set of addressing modes. The register field specifies a register for the mode or a submode for modes that do not use registers.

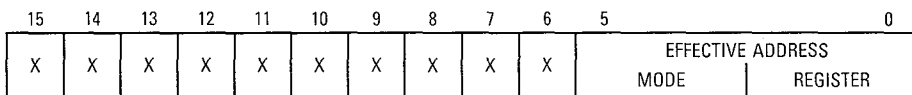


Figure 2-3. Single Effective Address Instruction Operation Word

Many instructions imply the addressing mode for one of the operands. The formats of these instructions include appropriate fields for operands that use only one addressing mode.

The effective address field may require additional information to fully specify the operand address. This additional information, called the effective address extension, is contained in an additional word or words and is considered part of the instruction. Refer to **2.5 EFFECTIVE ADDRESS ENCODING SUMMARY** for a description of the extension word formats.

The notational conventions used in the addressing mode descriptions in this section are:

EA—Effective address

An—Address register n

Example: A3 is address register 3

Dn—Data register n

Example: D5 is data register 5

Xn.SIZE*SCALE—Denotes index register n (data or address), the index size (W for word, L for long word), and a scale factor (1, 2, 4, or 8, for no, word, long-word or 8 for quad-word scaling, respectively).

PC—The program counter

d_n—Displacement value, n bits wide

bd—Base displacement

od—Outer displacement

L—Long-word size

W—Word size

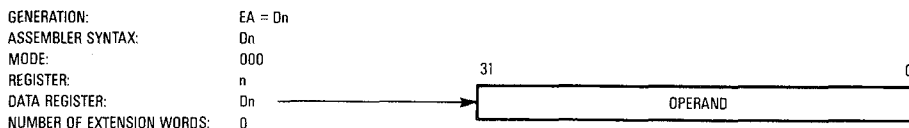
()—Identify an indirect address in a register

[]—Identify an indirect address in memory

When the addressing mode uses a register, the register field of the operation word specifies the register to be used. Other fields within the instruction specify whether the register selected is an address or data register and how the register is to be used.

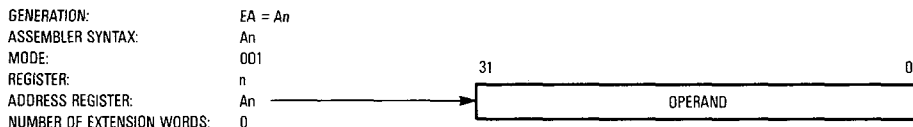
2.4.1 Data Register Direct Mode

In the data register direct mode, the operand is in the data register specified by the effective address register field.



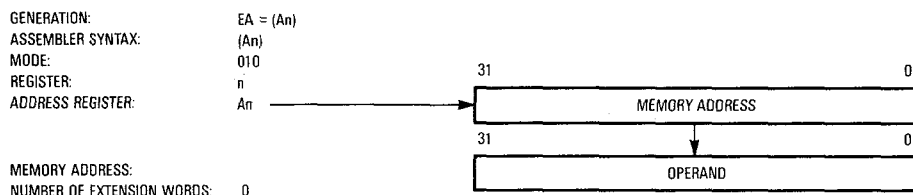
2.4.2 Address Register Direct Mode

In the address register direct mode, the operand is in the address register specified by the effective address register field.



2.4.3 Address Register Indirect Mode

In the address register indirect mode, the operand is in memory, and the address of the operand is in the address register specified by the register field.



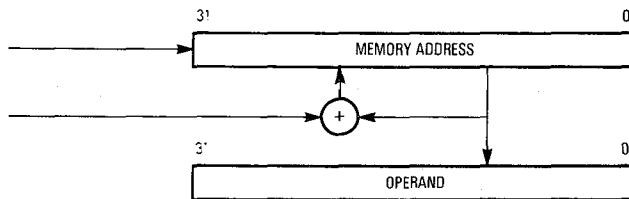
2.4.4 Address Register Indirect with Postincrement Mode

In the address register indirect with postincrement mode, the operand is in memory, and the address of the operand is in the address register specified by the register field. After the operand address is used, it is incremented by one, two, or four depending on the size of the operand: byte, word, or long word. Coprocessors may support incrementing for any size of operand up to 255 bytes. If the address register is the stack pointer and the operand size is byte, the address is incremented by two rather than one to keep the stack pointer aligned to a word boundary.

GENERATION: $EA = (An)$
 $An = An + SIZE$
 ASSEMBLER SYNTAX: $(An) +$
 MODE: 011
 REGISTER: n
 ADDRESS REGISTER: An

 OPERAND LENGTH (1, 2, OR 4):

 MEMORY ADDRESS:
 NUMBER OF EXTENSION WORDS: 0



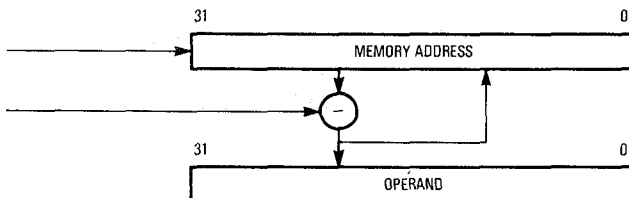
2.4.5 Address Register Indirect with Predecrement Mode

In the address register indirect with predecrement mode, the operand is in memory, and the address of the operand is in the address register specified by the register field. Before the operand address is used, it is decremented by one, two, or four depending on the operand size: byte, word, or long word. Coprocessors may support decrementing for any operand size up to 255 bytes. If the address register is the stack pointer and the operand size is byte, the address is decremented by two rather than one to keep the stack pointer aligned to a word boundary.

GENERATION: $An = An - SIZE$
 $EA = (An)$
 ASSEMBLER SYNTAX: $-(An)$
 MODE: 100
 REGISTER: n
 ADDRESS REGISTER: An

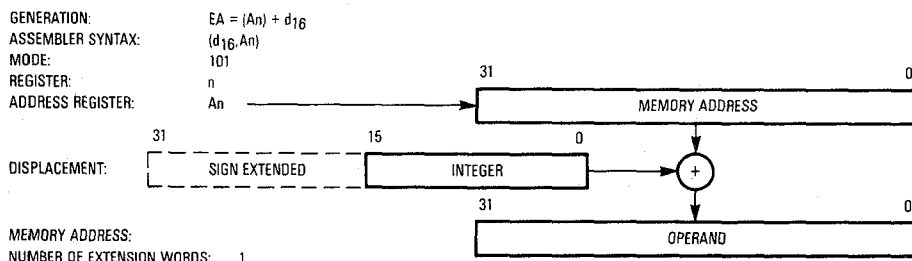
 OPERAND LENGTH (1, 2, OR 4):

 MEMORY ADDRESS:
 NUMBER OF EXTENSION WORDS: 0



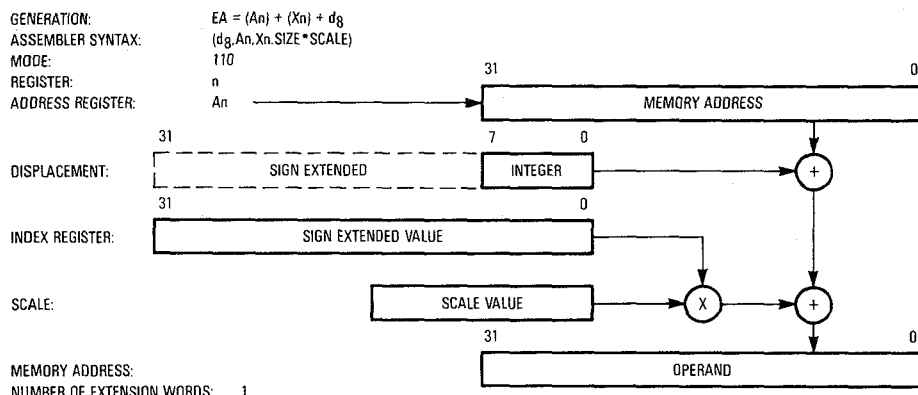
2.4.6 Address Register Indirect with Displacement Mode

In the address register indirect with displacement mode, the operand is in memory. The address of the operand is the sum of the address in the address register plus the sign-extended 16-bit displacement integer in the extension word. Displacements are always sign-extended to 32 bits prior to being used in effective address calculations.



2.4.7 Address Register Indirect with Index (8-Bit Displacement) Mode

This addressing mode requires one extension word that contains the index register indicator and an 8-bit displacement. The index register indicator includes size and scale information. In this mode, the operand is in memory. The address of the operand is the sum of the contents of the address register, the sign-extended displacement value in the low-order eight bits of the extension word, and the sign-extended contents of the index register (possibly scaled). The user must specify the displacement, the address register, and the index register in this mode.

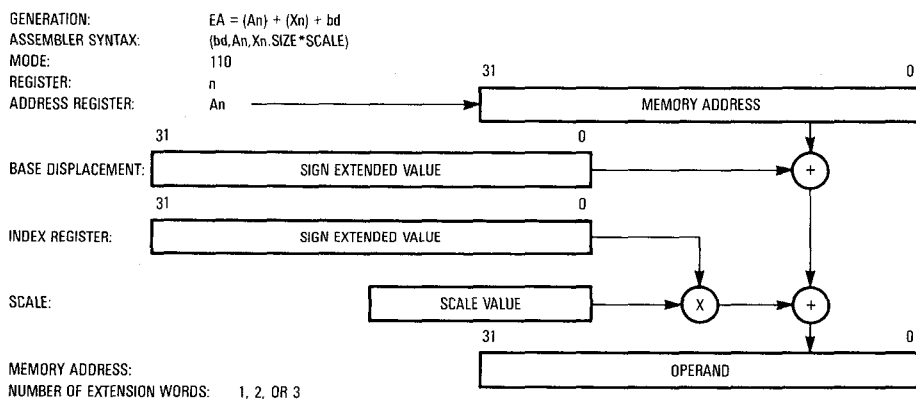


2.4.8 Address Register Indirect with Index (Base Displacement) Mode

This addressing mode requires an index register indicator and an optional 16- or 32-bit sign-extended base displacement. The index register indicator includes size and scaling information. The operand is in memory. The address of the operand is the sum of the contents of the address register, the scaled contents of the sign-extended index register, and the base displacement.

2

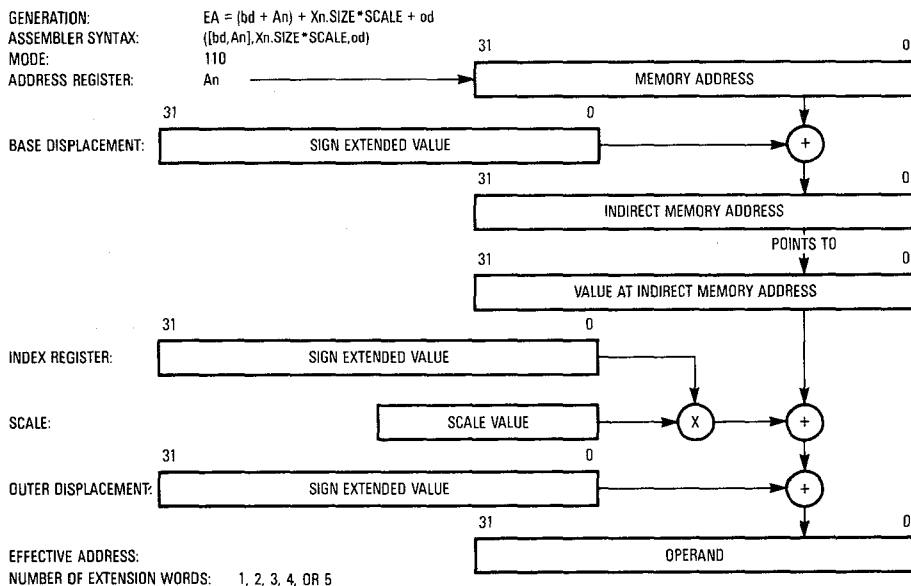
In this mode, the address register, the index register, and the displacement are all optional. If none is specified, the effective address is zero. This mode provides a data register indirect address when no address register is specified and the index register is a data register (Dn).



2.4.9 Memory Indirect Postindexed Mode

In this mode, the operand and its address are in memory. The processor calculates an intermediate indirect memory address using the base register (A_n) and base displacement (bd). The processor accesses a long word at this address and adds the index operand ($X_n.SIZE * SCALE$) and the outer displacement to yield the effective address. Both displacements and the index register contents are sign-extended to 32 bits.

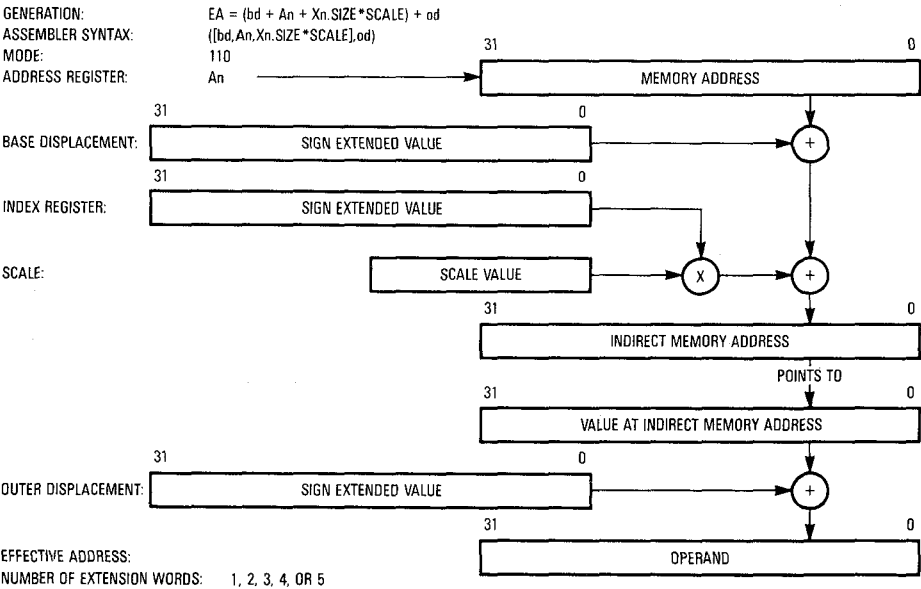
In the syntax for this mode, brackets enclose the values used to calculate the intermediate memory address. All four user-specified values are optional. Both the base and outer displacements may be null, word, or long word. When a displacement is omitted or an element is suppressed, its value is taken as zero in the effective address calculation.



2.4.10 Memory Indirect Preindexed Mode

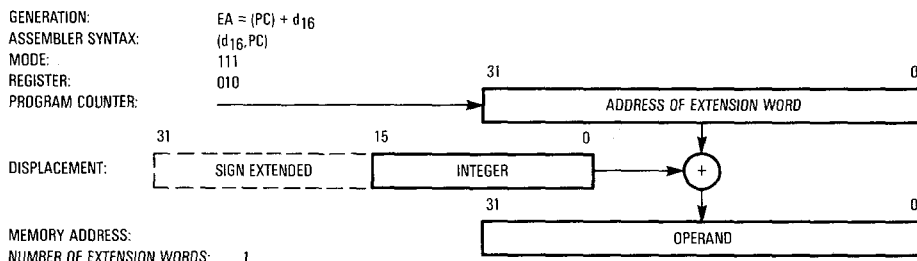
In this mode, the operand and its address are in memory. The processor calculates an intermediate indirect memory address using the base register (An), a base displacement (bd), and the index operand (Xn.SIZE * SCALE). The processor accesses a long word at this address and adds the outer displacement to yield the effective address. Both displacements and the index register contents are sign-extended to 32 bits.

In the syntax for this mode, brackets enclose the values used to calculate the intermediate memory address. All four user-specified values are optional. Both the base and outer displacements may be null, word, or long word. When a displacement is omitted or an element is suppressed, its value is taken as zero in the effective address calculation.



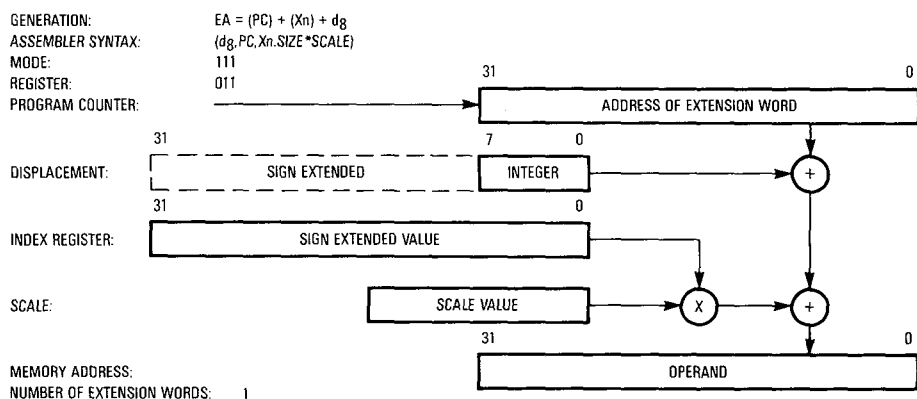
2.4.11 Program Counter Indirect with Displacement Mode

In this mode, the operand is in memory. The address of the operand is the sum of the address in the PC and the sign-extended 16-bit displacement integer in the extension word. The value in the PC is the address of the extension word. The reference is a program space reference and is only allowed for reads (refer to 4.2 ADDRESS SPACE TYPES).



2.4.12 Program Counter Indirect with Index (8-Bit Displacement) Mode

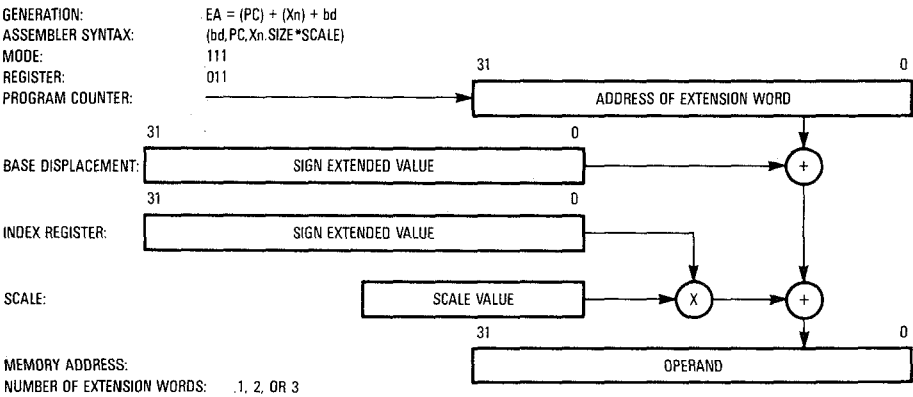
This mode is similar to the address register indirect with index (8-bit displacement) mode described in 2.4.7 Address Register Indirect with Index (8-Bit Displacement) Mode, but the PC is used as the base register. The operand is in memory. The address of the operand is the sum of the address in the PC, the sign-extended displacement integer in the lower eight bits of the extension word, and the sized, scaled, and sign-extended index operand. The value in the PC is the address of the extension word. This reference is a program space reference and is only allowed for reads. The user must include the displacement, the PC, and the index register when specifying this addressing mode.



2.4.13 Program Counter Indirect with Index (Base Displacement) Mode

This mode is similar to the address register indirect with index (base displacement) mode described in **2.4.8 Address Register Indirect with Index (Base Displacement) Mode**, but the PC is used as the base register. It requires an index register indicator and an optional 16- or 32-bit sign-extended base displacement. The operand is in memory. The address of the operand is the sum of the contents of the PC, the scaled contents of the sign-extended index register, and the base displacement. The value of the PC is the address of the first extension word. The reference is a program space reference and is only allowed for reads (refer to **4.2 ADDRESS SPACE TYPES**).

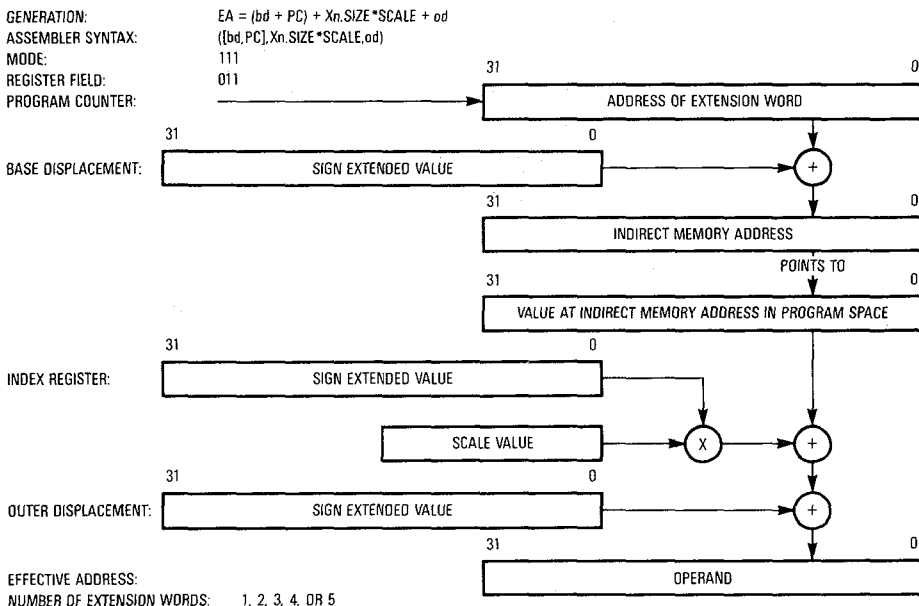
In this mode, the PC, the index register, and the displacement are all optional. However, the user must supply the assembler notation "ZPC" (zero value is taken for the PC) to indicate that the PC is not used. This allows the user to access the program space without using the PC in calculating the effective address. The user can access the program space with a data register indirect access by placing ZPC in the instruction and specifying a data register (Dn) as the index register.



2.4.14 Program Counter Memory Indirect Postindexed Mode

This mode is similar to the memory indirect postindexed mode described in **2.4.9 Memory Indirect Postindexed Mode**, but the PC is used as the base register. Both the operand and operand address are in memory. The processor calculates an intermediate indirect memory address by adding a base displacement (bd) to the PC contents. The processor accesses a long word at that address and adds the scaled contents of the index register and the optional outer displacement (od) to yield the effective address. The value of the PC used in the calculation is the address of the first extension word. The reference is a program space reference and is only allowed for reads (refer to **4.2 ADDRESS SPACE TYPES**).

In the syntax for this mode, brackets enclose the values used to calculate the intermediate memory address. All four user-specified values are optional. However, the user must supply the assembler notation ZPC (zero value is taken for the PC) to indicate that the PC is not used. This allows the user to access the program space without using the PC in calculating the effective address. Both the base and outer displacements may be null, word, or long word. When a displacement is omitted or an element is suppressed, its value is taken as zero in the effective address calculation.

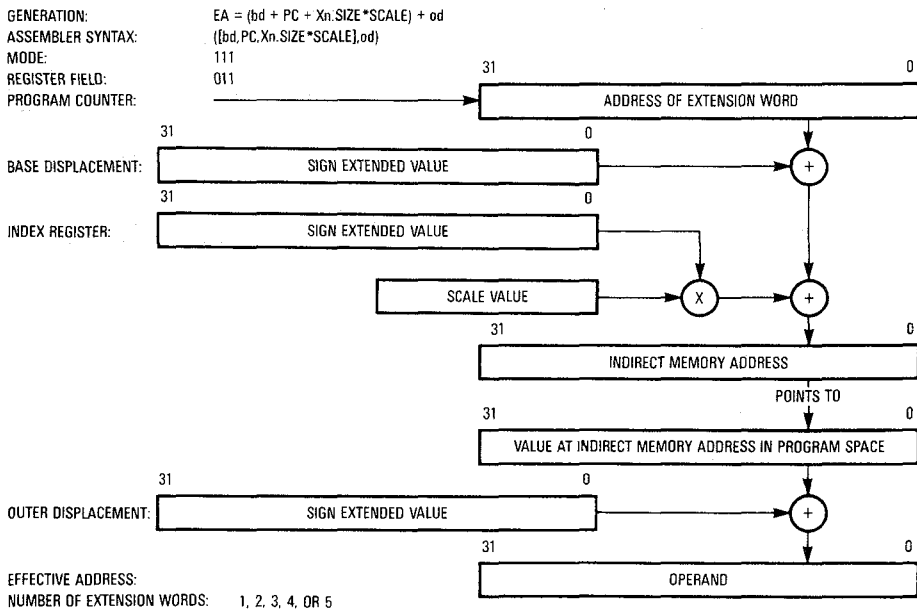


2.4.15 Program Counter Memory Indirect Preindexed Mode

This mode is similar to the memory indirect preindexed mode described in **2.4.10 Memory Indirect Preindexed Mode**, but the PC is used as the base register. Both the operand and operand address are in memory. The processor calculates an intermediate indirect memory address by adding the PC contents, a base displacement (bd), and the scaled contents of an index register. The processor accesses a long word at that address and adds the optional outer displacement (od) to yield the effective address. The value of the PC is the address of the first extension word. The reference is a program space reference and is only allowed for reads (refer to **4.2 ADDRESS SPACE TYPES**).

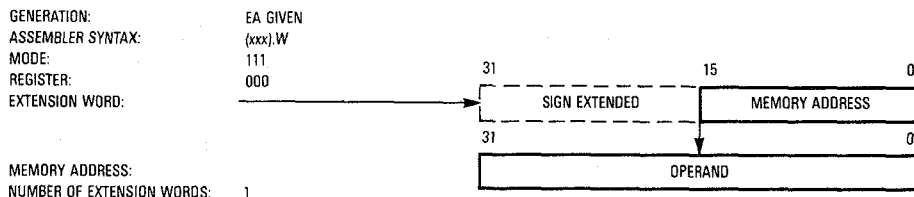
2

In the syntax for this mode, brackets enclose the values used to calculate the intermediate memory address. All four user-specified values are optional. However, the user must supply the assembler notation ZPC (zero value is taken for the PC) to indicate that the PC is not used. This allows the user to access the program space without using the PC in calculating the effective address. Both the base and outer displacements may be null, word, or long word. When a displacement is omitted or an element is suppressed, its value is taken as zero in the effective address calculation.



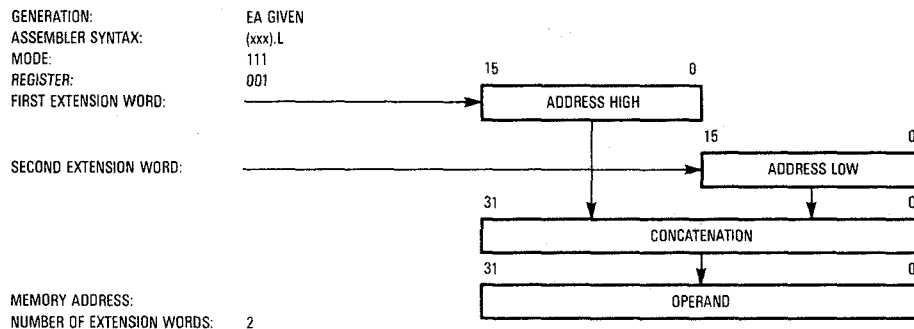
2.4.16 Absolute Short Addressing Mode

In this addressing mode, the operand is in memory, and the address of the operand is in the extension word. The 16-bit address is sign-extended to 32 bits before it is used.



2.4.17 Absolute Long Addressing Mode

In this mode, the operand is in memory, and the address of the operand occupies the two extension words following the instruction word in memory. The first extension word contains the high-order part of the address; the low-order part of the address is the second extension word.





2.4.18 Immediate Data

In this addressing mode, the operand is in one or two extension words:

Byte Operation

Operand is in the low-order byte of the extension word

2

Word Operation

Operand is in the extension word

Long-Word Operation

The high-order 16 bits of the operand are in the first extension word;
the low-order 16 bits are in the second extension word.

Coprocessor instructions can support immediate data of any size. The instruction word is followed by as many extension words as are required.

Generation:	Operand given
Assembler Syntax:	#xxx
Mode Field:	111
Register Field:	100
Number of Extension Words:	1 or 2, except for coprocessor instructions

2.5 EFFECTIVE ADDRESS ENCODING SUMMARY

Most of the addressing modes use one of the three formats shown in Figure 2-4. The single effective address instruction is in the format of the instruction word. The encoding of the mode field of this word selects the addressing mode. The register field contains the general register number or a value that selects the addressing mode when the mode field contains “111”. Table 2-2 shows the encoding of these fields. Some indexed or indirect modes use the instruction word followed by the brief format extension word. Other indexed or indirect modes consist of the instruction word and the full format of extension words. The longest instruction for the MC68030 contains 10 extension words. It is a MOVE instruction with full format extension words for both the source and destination effective addresses and with 32-bit base displacements and 32-bit outer displacements for both addresses. However, coprocessor instructions can have any number of extension words. Refer to the coprocessor instruction formats in **SECTION 10 COPROCESSOR INTERFACE DESCRIPTION**.

For effective addresses that use the full format, the index suppress (IS) bit and the index/indirect selection (I/IS) field determine the type of indexing and indirection. Table 2-1 lists the indexing and indirection operations corresponding to all combinations of IS and I/IS values.

Table 2-1. IS-I/IS Memory Indirection Encodings

IS	Index/Indirect	Operation
0	000	No Memory Indirection
0	001	Indirect Preindexed with Null Outer Displacement
0	010	Indirect Preindexed with Word Outer Displacement
0	011	Indirect Preindexed with Long Outer Displacement
0	100	Reserved
0	101	Indirect Postindexed with Null Outer Displacement
0	110	Indirect Postindexed with Word Outer Displacement
0	111	Indirect Postindexed with Long Outer Displacement
1	000	No Memory Indirection
1	001	Memory Indirect with Null Outer Displacement
1	010	Memory Indirect with Word Outer Displacement
1	011	Memory Indirect with Long Outer Displacement
1	100–111	Reserved

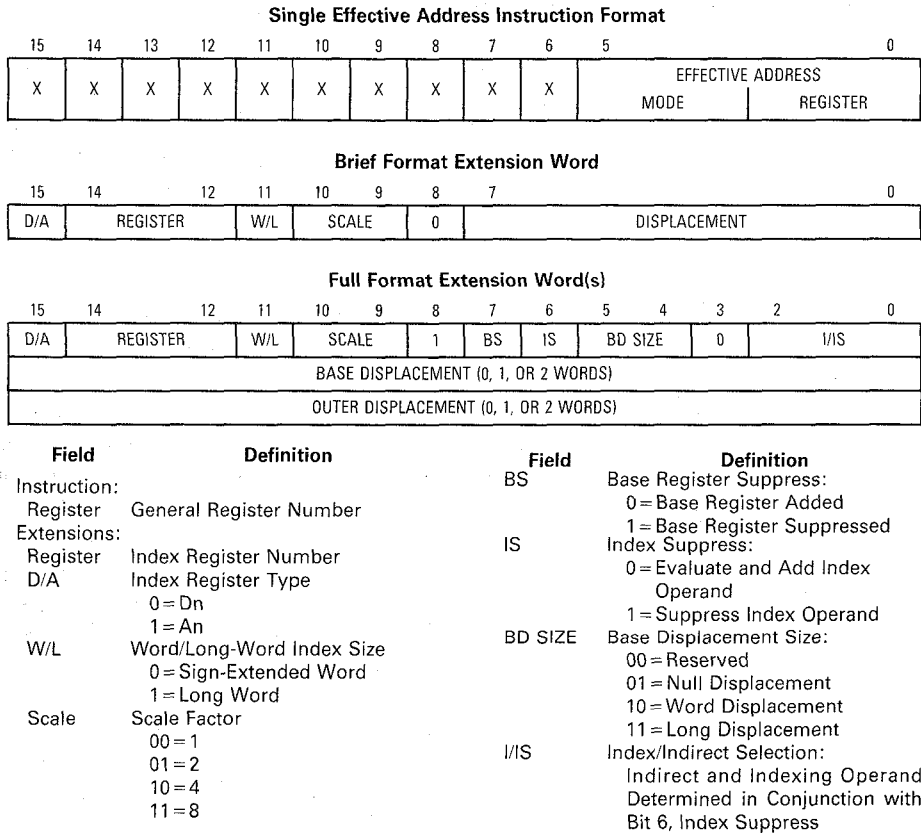


Figure 2-4. Effective Address Specification Formats

Effective address modes are grouped according to the use of the mode. They can be classified as follows:

- Data
A data addressing effective address mode is one that refers to data operands.
- Memory
A memory addressing effective address mode is one that refers to memory operands.
- Alterable
An alterable addressing effective address mode is one that refers to alterable (writable) operands.
- Control
A control addressing effective address mode is one that refers to memory operands without an associated size.

Table 2-2 shows the categories to which each of the effective addressing modes belong.

Table 2-2. Effective Addressing Mode Categories

Address Modes	Mode	Register	Data	Memory	Control	Alterable	Assembler Syntax
Data Register Direct	000	reg. no.	X	—	—	X	Dn
Address Register Direct	001	reg. no.	—	—	—	X	An
Address Register Indirect	010	reg. no.	X	X	X	X	(An)
Address Register Indirect with Postincrement	011	reg. no.	X	X	—	X	(An) +
Address Register Indirect with Predecrement	100	reg. no.	X	X	—	X	-- (An)
Address Register Indirect with Displacement	101	reg. no.	X	X	X	X	(d ₁₆ ,An)
Address Register Indirect with Index (8-Bit Displacement)	110	reg. no.	X	X	X	X	(dg,An,Xn)
Address Register Indirect with Index (Base Displacement)	110	reg. no.	X	X	X	X	{bd,An,Xn}
Memory Indirect Postindexed	110	reg. no.	X	X	X	X	{[bd,An],Xn,od}
Memory Indirect Preindexed	110	reg. no.	X	X	X	X	{[bd,An,Xn],od}
Absolute Short	111	000	X	X	X	X	(xxx).W
Absolute Long	111	001	X	X	X	X	(xxx).L
Program Counter Indirect with Displacement	111	010	X	X	X	—	(d ₁₆ ,PC)
Program Counter Indirect with Index (8-Bit) Displacement	111	011	X	X	X	—	(dg,PC,Xn)
Program Counter Indirect with Index (Base Displacement)	111	011	X	X	X	—	{bd,PC,Xn}
PC Memory Indirect Postindexed	111	011	X	X	X	—	{[bd,PC],Xn,od}
PC Memory Indirect Preindexed	111	011	X	X	X	—	{[bd,PC,Xn],od}
Immediate	111	100	X	X	—	—	#{data}

These categories are sometimes combined, forming new categories that are more restrictive. Two combined classifications are alterable memory or data alterable. The former refers to those addressing modes that are both alterable and memory addresses, and the latter refers to addressing modes that are both data and alterable.

2.6 PROGRAMMER'S VIEW OF ADDRESSING MODES

Extensions to the indexed addressing modes, indirection, and full 32-bit displacements provide additional programming capabilities for both the MC68020 and the MC68030. This section describes addressing techniques that exploit these capabilities and summarizes the addressing modes from a programming point of view.

Several of the addressing techniques described in this section use data registers and address registers interchangeably. While the MC68030 provides this capability, its performance has been optimized for addressing with address registers. The performance of a program that uses address registers in address calculations is superior to that of a program that similarly uses data registers. The specification of addresses with data registers should be used sparingly (if at all), particularly in programs that require maximum performance.

2.6.1 Addressing Capabilities

In both the MC68020 and the MC68030, setting the base register suppress (BS) bit in the full format extension word (see Figure 2-4) suppresses use of the base address register in calculating the effective address. This allows any index register to be used in place of the base register. Since any of the data registers can be index registers, this provides a data register indirect form (Dn). The mode could be called register indirect (Rn) since either a data register or an address register can be used. This addressing mode is an extension to the M68000 Family because the MC68030 and MC68020 can use both the data registers and the address registers to address memory. The capability of specifying the size and scale of an index register ($Xn.SIZE * SCALE$) in these modes provides additional addressing flexibility. Using the SIZE parameter, either the entire contents of the index register can be used, or the least significant word can be sign-extended to provide a 32-bit index value (refer to Figure 2-5).

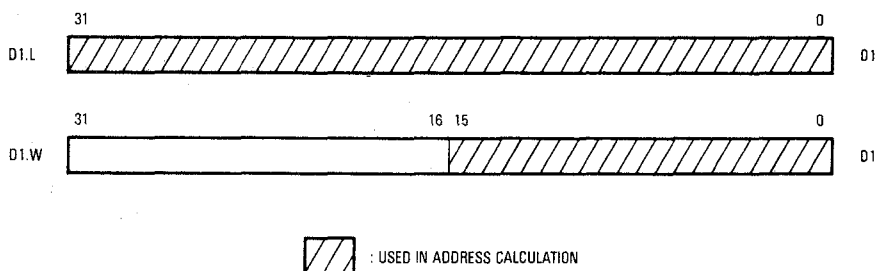


Figure 2-5. Using SIZE in the Index Selection

For both the MC68020 and the MC68030, the register indirect modes can be extended further. Since displacements can be 32 bits wide, they can represent absolute addresses or the results of expressions that contain absolute addresses. This allows the general register indirect form to be (bd, Rn) or (bd, An, Rn) when the base register is not suppressed. Thus, an absolute address can be directly indexed by one or two registers (refer to Figure 2-6).

SYNTAX: (bd, An, Rn)

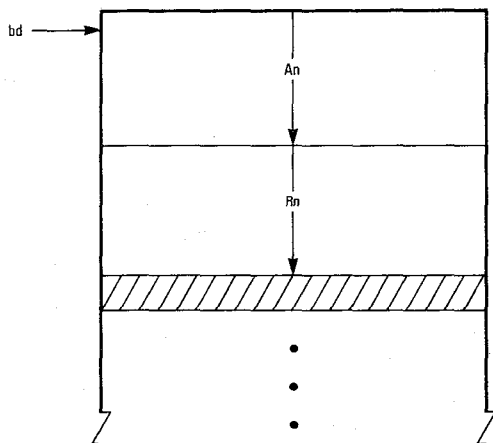


Figure 2-6. Using Absolute Address with Indexes

Scaling provides an optional shifting of the value in an index register to the left by zero, one, two, or three bits before using it in the effective address calculation (the actual value in the index register remains unchanged). This is equivalent to multiplying the register by one, two, four, or eight for direct subscripting into an array of elements of corresponding size using an arithmetic value residing in any of the 16 general registers. Scaling does not add to the effective address calculation time. However, when combined with the appropriate derived modes, it produces additional capabilities. Arrayed structures can be addressed absolutely and then subscripted, $(bd, Rn * scale)$, for example. Optionally, an address register that contains a dynamic displacement can be included in the address calculation $(bd, An, Rn * scale)$. Another variation that can be derived is $(An, Rn * scale)$. In the first case, the array address is the sum of the contents of a register and a displacement, as shown in Figure 2-7. In the second example, An contains the address of an array and Rn contains a subscript.

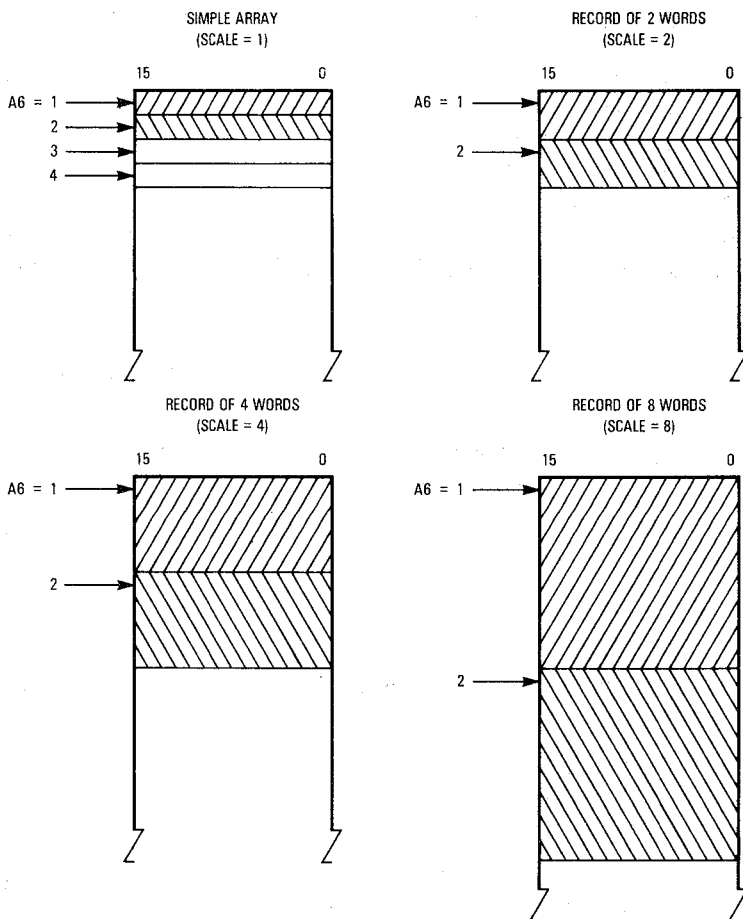
SYNTAX: MOVE.W (A5,A6.L*SCALE),(A7)

WHERE:

A5 = ADDRESS OF ARRAY STRUCTURE

A6 = INDEX NUMBER OF ARRAY ITEM

A7 = STACK POINTER



NOTE: Regardless of array structure, software increments index by the appropriate amount to point to next record.

Figure 2-7. Addressing Array Items

The memory indirect addressing modes use a long-word pointer in memory to access an operand. Any of the modes previously described can be used to address the memory pointer. Because the base and index registers can both be suppressed, the displacement acts as an absolute address, providing indirect absolute memory addressing (refer to Figure 2-8).

The outer displacement (od) available in the memory indirect modes is added to the pointer in memory. The syntax for these modes is $([bd, An], Xn, od)$ and $([bd, An, Xn], od)$. When the pointer is the address of a structure in memory and the outer displacement is the offset of an item in the structure, the memory indirect modes can access the item efficiently (refer to Figure 2-9).

Memory indirect addressing modes are used with a base displacement in five basic forms:

1. $[bd, An]$ — Indirect, suppressed index register
2. $([bd, An, Xn])$ — Preindexed indirect
3. $([bd, An], Xn)$ — Postindexed indirect
4. $([bd, An, Xn], od)$ — Preindexed indirect with outer displacement
5. $([bd, An], Xn, od)$ — Postindexed indirect with outer displacement

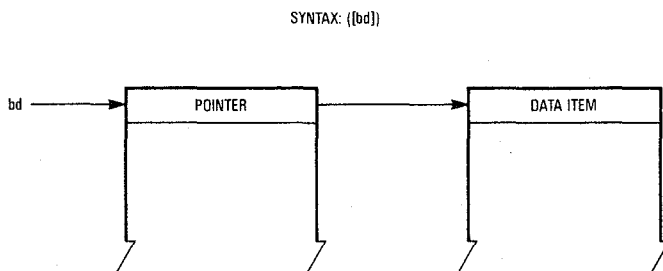


Figure 2-8. Using Indirect Absolute Memory Addressing

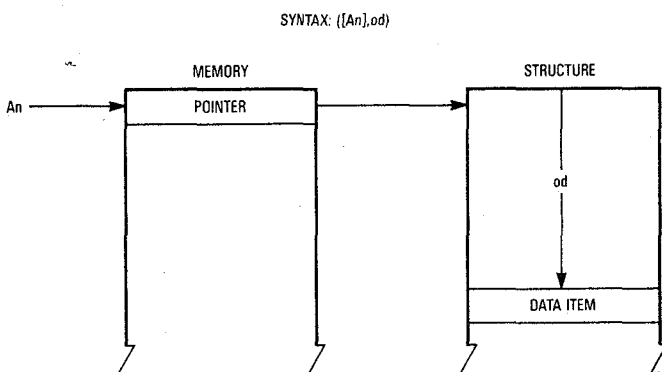


Figure 2-9. Accessing an Item in a Structure Using Pointer

The indirect, suppressed index register mode (see Figure 2-10) uses the contents of register An as an index to the pointer located at the address specified by the displacement. The actual data item is at the address in the selected pointer.

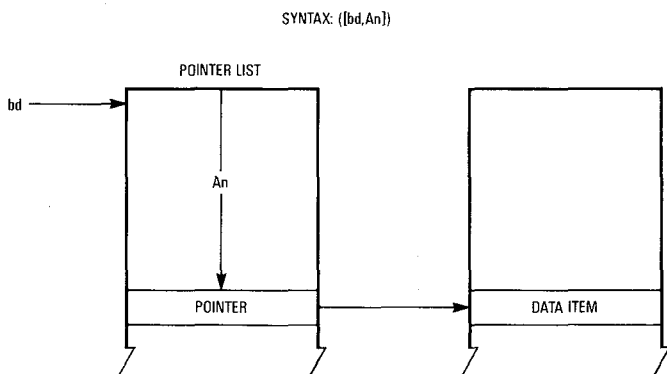


Figure 2-10. Indirect Addressing, Suppressed Index Register

The preindexed indirect mode (see Figure 2-11) uses the contents of An as an index to the pointer list structure at the displacement. Register Xn is the index to the pointer, which contains the address of the data item.

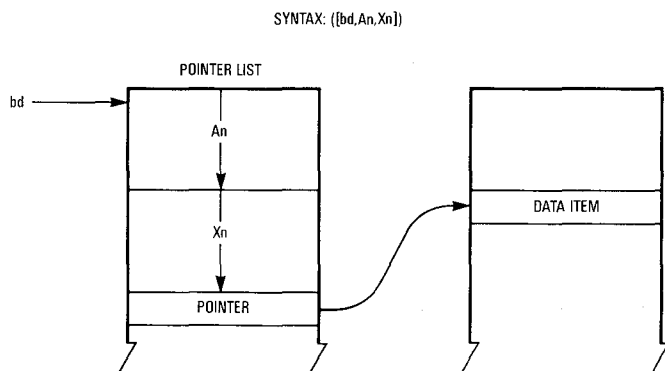


Figure 2-11. Preindexed Indirect Addressing

The postindexed indirect mode (see Figure 2-12) uses the contents of A_n as an index to the pointer list at the displacement. Register X_n is used as an index to the structure of data items located at the address specified by the pointer. Figure 2-13 shows the preindexed indirect addressing with outer displacement mode.

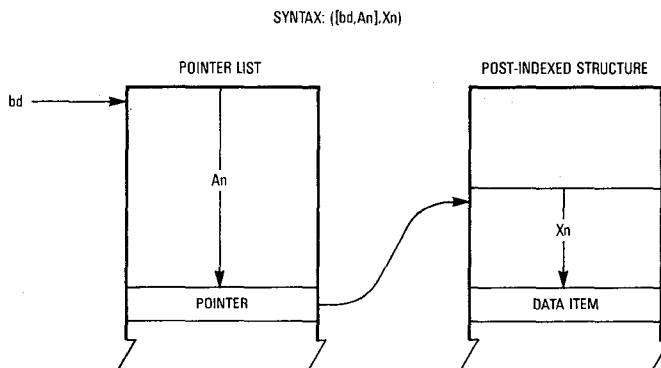


Figure 2-12. Postindexed Indirect Addressing

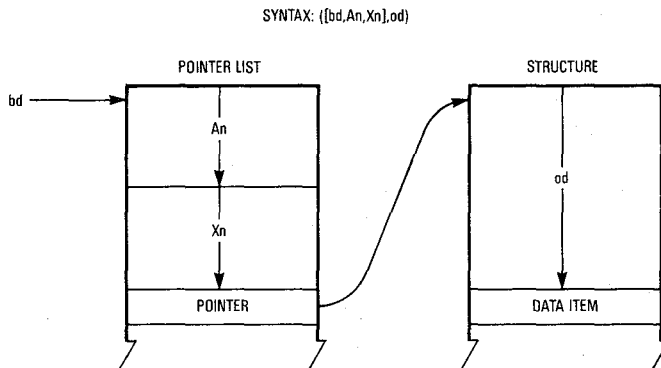


Figure 2-13. Preindexed Indirect Addressing with Outer Displacement

The postindexed indirect mode with outer displacement (see Figure 2-14) uses the contents of An as an index to the pointer list at the displacement. Register Xn is used as an index to the structure of data structures at the address in the pointer. The outer displacement (od) is the displacement of the data item within the selected data structure.

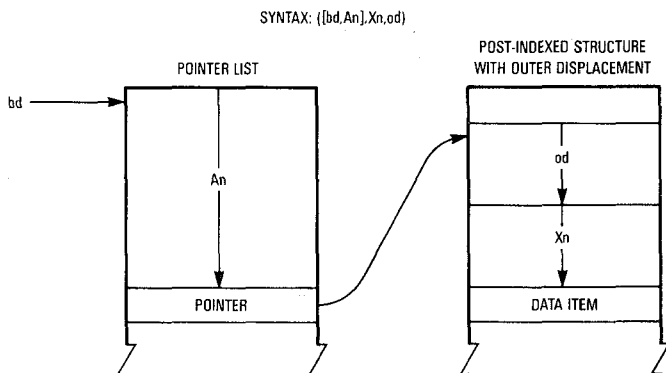


Figure 2-14. Postindexed Indirect Addressing with Outer Displacement

2.6.2 General Addressing Mode Summary

The addressing modes described in the previous section are derived from specific combinations of options in the indexing mode or a selection of two alternate addressing modes. For example, the addressing mode called register indirect (Rn) assembles as the address register indirect if the register is an address register. If Rn is a data register, the assembler uses the address register indirect with index mode using the data register as the indirect register and suppresses the address register by setting the base suppress bit in the effective address specification. Assigning an address register as Rn provides higher performance than using a data register as Rn . Another case is (bd, An) , which selects an addressing mode depending on the size of the displacement. If the displacement is 16 bits or less, the address register indirect with displacement mode ($d16, An$) is used. When a 32-bit displacement is required, the address register indirect with index (bd, An, Xn) is used with the index register suppressed.

It is useful to examine the derived addressing modes available to a programmer (without regard to the MC68030 effective addressing mode actually encoded) because the programmer need not be concerned about these decisions. The assembler can choose the more efficient addressing mode to encode.

In the list of derived addressing modes that follows, common programming terms are used. The following definitions apply:

pointer	— Long-word value in a register or in memory which represents an address.
base	— A pointer combined with a displacement to represent an address.
index	— A constant or variable value added into an effective address calculation. A constant index is a displacement. A variable index is always represented by a register containing the value.
disp	— Displacement, a constant index.
subscript	— The use of any of the data or address registers as a variable index subscript into arrays of items 1, 2, 4, or 8 bytes in size.
relative	— An address calculated from the program counter contents. The address is position independent and is in program space. All other addresses but psaddr are in data space.
addr	— An absolute address.
psaddr	— An absolute address in program space. All other addresses but PC relative are in data space.
preindexed	— All modes from absolute address through program counter relative.

- postindexed — Any of the following modes:
- addr — Absolute address in data space
 - psaddr,ZPC — Absolute address in program space
 - An — Register pointer
 - disp,An — Register pointer with constant displacement
 - addr,An — Absolute address with single variable name
 - disp,PC — Simple PC relative

The addressing modes defined in programming terms, which are derivations of the addressing modes provided by the MC68030 architecture, are as follows:

Immediate Data — #data:

The data is a constant located in the instruction stream.

Register Direct — Rn:

The contents of a register contain the operand.

Scanning Modes:

(An)+

Address register pointer automatically incremented after use.

–(An)

Address register pointer automatically decremented before use.

Absolute Address:

(addr)

Absolute address in data space.

(psaddr,ZPC)

Absolute address in program space. Symbol ZPC suppresses the PC, but retains PC relative mode to directly access the program space.

Register Pointer:

(Rn)

Register as a pointer.

(disp,Rn)

Register as a pointer with constant index (or base address).

Indexing:

(An,Rn)

Register pointer An with variable index Rn.

(disp,An,Rn)

Register pointer with constant and variable index (or a base address with a variable index).

(addr,Rn)

Absolute address with variable index.

(addr,An,Rn)

Absolute address with two variable indexes.

Subscripting:

(An,Rn*scale)

Address register pointer subscript.

(disp,An,Rn*scale)

Address register pointer subscript with constant displacement (or base address with subscript).

(addr, Rn*scale)

Absolute address with subscript.

(addr,An,Rn*scale)

Absolute address subscript with variable index.

Program Relative:

(disp,PC)

Simple PC relative.

(disp,PC,Rn)

PC relative with variable index.

(disp,PC,Rn*scale)

PC relative with subscript.

Memory Pointer:

`([preindexed])`

Memory pointer directly to data operand.

`([preindexed],disp)`

Memory pointer as base with displacement to data operand.

`([postindexed],Rn)`

Memory pointer with variable index.

`([postindexed],disp,Rn)`

Memory pointer with constant and variable index.

`([postindexed],Rn*scale)`

Memory pointer subscripted.

`([postindexed], disp, Rn*scale)`

Memory pointer subscripted with constant index.

2.1 M68000 FAMILY ADDRESSING COMPATIBILITY

Programs can be easily transported from one member of the M68000 Family to another in an upward compatible fashion. The user object code of each early member of the family is upward compatible with newer members and can be executed on the newer microprocessor without change. The address extension word(s) are encoded with the information that allows the MC68020/MC68030 to distinguish the new address extensions to the basic M68000 Family architecture. The address extension words for the early MC68000/MC68008/MC68010 microprocessors and for the newer 32-bit MC68020/MC68030 microprocessors are shown in Figure 2-15. Notice the encoding for SCALE used by the MC68020/MC68030 is a compatible extension of the M68000 architecture. A value of zero for SCALE is the same encoding for both extension words; hence, software that uses this encoding is both upward and downward compatible across all processors in the product line. However, the other values of SCALE are not found in both extension formats; thus, while software can be easily migrated in an upward compatible direction, only nonscaled addressing is supported in a downward fashion. If the MC68000 were to execute an instruction that encoded a scaling factor, the scaling factor would be ignored and not access the desired memory address. The earlier microprocessors have no knowledge of the extension word formats implemented by newer processors; while they do detect illegal instructions, they do not decode invalid encodings of the extension words as exceptions.

2.8 OTHER DATA STRUCTURES

Stacks and queues are widely used data structures. The MC68030 implements a system stack and also provides instructions that support the use of user stacks and queues.

2.8.1 System Stack

Address register seven (A7) is used as the system stack pointer (SP). Any of the three system stack registers is active at any one time. The M and S bits of the status register determine which stack pointer is used. When S=0 indicating user mode (user privilege level), the user stack pointer (USP) is the active system stack pointer, and the master and interrupt stack pointers cannot be referenced. When S=1 indicating supervisor mode (at supervisor privilege level) and M=1, the master stack pointer (MSP) is the active system stack pointer. When S=1 and M=0, the interrupt stack pointer (ISP) is the active system stack pointer. This mode is the MC68030 default mode after reset and corresponds to the MC68000, MC68008, and MC68010 supervisor

MC68000/MC68008/MC68010 Address Extension Word

15	14	12	11	10	9	8	7	0
D/A	REGISTER	W/L	0	0	0	DISPLACEMENT INTEGER		

D/A: 0 = Data Register Select
 1 = Address Register Select
 W/L: 0 = Word-Sized Operation
 1 = Long-Word-Sized Operation

MC68020/MC68030 Address Extension Word

15	14	12	11	10	9	8	7	0
D/A	REGISTER	W/L	SCALE	0	DISPLACEMENT INTEGER			

D/A: 0 = Data Register Select
 1 = Address Register Select
 W/L: 0 = Word-Sized Operation
 1 = Long-Word-Sized Operation
 SCALE: 00 = Scale Factor 1 (Compatible with MC68000)
 01 = Scale Factor 2 (Extension to MC68000)
 10 = Scale Factor 4 (Extension to MC68000)
 11 = Scale Factor 8 (Extension to MC68000)

Figure 2-15. M68000 Family Address Extension Words

mode. The term supervisor stack pointer (SSP) refers to the master or interrupt stack pointers, depending on the state of the M bit. When M=1, the term SSP (or A7) refers to the MSP address register. When M=0, the term SSP (or A7) refers to the ISP address register. The active system stack pointer is implicitly referenced by all instructions that use the system stack. Each system stack fills from high to low memory.

A subroutine call saves the program counter on the active system stack, and the return restores it from the active system stack. During the processing of traps and interrupts, both the program counter and the status register are saved on the supervisor stack (either master or interrupt). Thus, the execution of supervisor code is independent of user code and the condition of the user stack; conversely, user programs use the user stack pointer independently of supervisor stack requirements.

To keep data on the system stack aligned for maximum efficiency, the active stack pointer is automatically decremented or incremented by two for all byte-sized operands moved to or from the stack. In long-word-organized

memory, aligning the stack pointer on a long-word address significantly increases the efficiency of stacking exception frames, subroutine calls and returns, and other stacking operations.

2 2.8.2 User Program Stacks

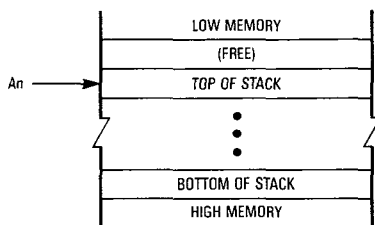
The user can implement stacks with the address register indirect with post-increment and predecrement addressing modes. With address register An ($n=0-6$), the user can implement a stack that is filled either from high to low memory or from low to high memory. Important considerations are:

- Use the predecrement mode to decrement the register before its contents are used as the pointer to the stack.
- Use the postincrement mode to increment the register after its contents are used as the pointer to the stack.
- Maintain the stack pointer correctly when byte, word, and long-word items are mixed in these stacks.

To implement stack growth from high to low memory, use:

- (An) to push data on the stack,
- $(An)+$ to pull data from the stack.

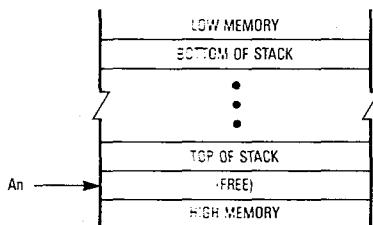
For this type of stack, after either a push or a pull operation, register An points to the top item on the stack. This is illustrated as:



To implement stack growth from low to high memory, use:

- $(An)+$ to push data on the stack,
- (An) to pull data from the stack.

In this case, after either a push or pull operation, register An points to the next available space on the stack. This is illustrated as:



2.8.3 Queues

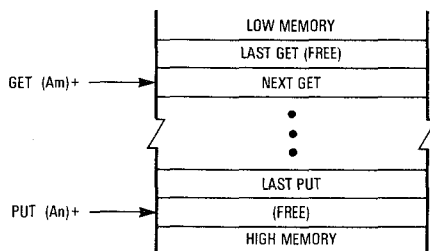
The user can implement queues with the address register indirect with post-increment or predecrement addressing modes. Using a pair of address registers (two of A0–A6), the user can implement a queue which is filled either from high to low memory or from low to high memory. Two registers are used because queues are pushed from one end and pulled from the other. One register, An, contains the “put” pointer; the other, Am, the “get” pointer.

To implement growth of the queue from low to high memory, use:

$(An) +$ to put data into the queue,

$(Am) +$ to get data from the queue.

After a “put” operation, the “put” address register points to the next available space in the queue, and the unchanged “get” address register points to the next item to be removed from the queue. After a “get” operation, the “get” address register points to the next item to be removed from the queue, and the unchanged “put” address register points to the next available space in the queue. This is illustrated as:

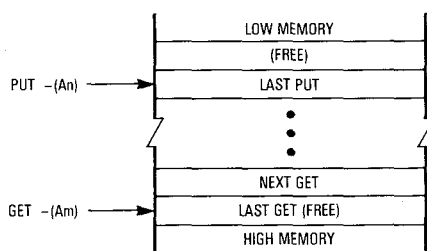


To implement the queue as a circular buffer, the relevant address register should be checked and adjusted, if necessary, before performing the “put” or “get” operation. The address register is adjusted by subtracting the buffer length (in bytes) from the register.

To implement growth of the queue from high to low memory, use:

- (An) to put data into the queue,
- (Am) to get data from the queue.

After a “put” operation, the “put” address register points to the last item placed in the queue, and the unchanged “get” address register points to the last item removed from the queue. After a “get” operation, the “get” address register points to the last item removed from the queue, and the unchanged “put” address register points to the last item placed in the queue. This is illustrated as:



To implement the queue as a circular buffer, the “get” or “put” operation should be performed first, and then the relevant address register should be checked and adjusted, if necessary. The address register is adjusted by adding the buffer length (in bytes) to the register contents.